

CKA: Install "Kubernetes the kubeadm way"

KodeKloud Complete Study Notes — Certified Kubernetes Administrator

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Demo Deployment with Kubeadm

This guide explains how to bootstrap a Kubernetes cluster using kubeadm with a master and two worker nodes.

In this guide, we'll walk through bootstrapping a Kubernetes cluster using kubeadm. The setup involves three virtual machines (VMs): one control plane (master) node and two worker nodes. We will review the VM network configurations, install the container runtime and Kubernetes components, initialize the control plane, deploy a pod network add-on, and finally join the worker nodes to complete the cluster.

1. VM Overview and Network Interfaces

Before you start, ensure that all required VMs are created. The cluster consists of one master and two worker nodes. Verify the network interfaces on each node by executing the `ip add` command.

Master Node Network Configuration

Run the following command on the master node:

```
``bash theme={null} vagrant@kubemaster:~$ ip add 1: lo: mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000 link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 inet 127.0.0.1/8 scope host lo valid_lft forever preferred_lft forever inet6 ::1/128 scope host valid_lft forever preferred_lft forever 2: enp0s3: mtu 1500 qdisc fq_codel state UP group default qlen 1000 link/ether 02:95:21:8a:38:bd brd ff:ff:ff:ff:ff:ff inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3 valid_lft forever preferred_lft 51sec inet6 fe80::95:21ff:fe8a:38bd/64 scope link valid_lft forever preferred_lft forever 3: enp0s8: mtu 1500 qdisc fq_codel
```

```
state UP group default qlen 1000 link/ether 02:42:fd:69:82:cd brd ff:ff:ff:ff:ff:ff inet 192.168.44.2/24 brd 192.168.44.255 scope global enp0s8 valid_lft forever preferred_lft forever inet6 fe80::42:fdff:fe69:82cd/64 scope link valid_lft forever preferred_lft forever
```

```
vagrant@kubemaster:~$ ls vagrant@kubemaster:~$
```

Worker Node One Network Configuration

On the first worker node, run:

```
```bash theme={null}
vagrant@kubenode01:~$ ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
 link/loopback 00:00:00:00:00:00
 inet 127.0.0.1/8 scope host lo
 valid_lft forever preferred_lft forever
 inet6 ::1/128 scope host
 valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
 link/ether 02:95:e1:8a:38:bd brd ff:ff:ff:ff:ff:ff
 inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
 valid_lft 83015sec preferred_lft 0sec
 inet6 fe80::a00:27ff:fe47:a2e/64 scope link
 valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
 link/ether 02:95:e1:8a:38:bd brd ff:ff:ff:ff:ff:ff
 inet 192.168.56.101/24 metric 1024 scope global enp0s8
 valid_lft forever preferred_lft forever

vagrant@kubenode01:~$
```

## Worker Node Two Network Configuration

On the second worker node, verify the network configuration:

```
```bash theme={null} vagrant@kubenode02:~$ ip add 1: lo: mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000 link/loopback 00:00:00:00:00:00 inet 127.0.0.1/8 scope host lo valid_lft forever preferred_lft forever inet6 ::1/128 scope link valid_lft forever preferred_lft forever 2: enp03: mtu 1500 qdisc fq_codel state UP group default qlen 1000 link/ether 02:95:e1:8a:38:b0 brd ff:ff:ff:ff:ff:ff inet 10.12.0.2/24 brd 10.12.0.255 scope global dynamic enp0s3 valid_lft 83155sec inet6 fe80::f50:e:38b:d6c5/64 scope link valid_lft forever preferred_lft forever 3: enp08: mtu 1500 qdisc fq_codel state UP group default qlen 1000 link/ether 02:80:2c:50:5d:f0 brd ff:ff:ff:ff:ff:ff inet6 fe80::27f:f50:5c0:5c0/64 scope link valid_lft forever preferred_lft forever
vagrant@kubenode02:~$
```

The output confirms that each node has the correct network interfaces and associated IP addresses, ensuring

> ![The image shows a webpage from Kubernetes documentation detailing the installation of kubeadm, kubelet

2. Reviewing Prerequisites

Before initializing your Kubernetes cluster, verify the following prerequisites:

- * A supported Linux distribution (e.g., Ubuntu).
- * A minimum of 2 GB memory and at least two CPUs per node.
- * Required kernel modules (BR, netfilter, overlay) are loaded.

<Callout icon="lightbulb" color="#1CB2FE">

Ensure that system variables are correctly set (to 1) so the network interfaces function properly. For more details, refer to the official [Kubernetes

Documentation](https://kubernetes.io/docs/setup/production-environment/container-runtimes/).

</Callout>

3. Installing the Container Runtime (ContainerD)

A container runtime is essential on every node. In this example, we will use ContainerD.

Step 1: Add the Kubernetes Repository and GPG Key

Execute these commands on all nodes:

```
```bash
theme={null}
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gnupg
Create the keyrings directory if it does not exist
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core/stable/v1.31/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/
echo "deb [signed-by=/etc/apt/keyrings/kubernetes.gpg] https://pkgs.k8s.io/core/stable/v1.31/deb/" | sudo
sudo apt-get update
sudo apt-get install -y kubelet kubectl
```

## Step 2: Install ContainerD

Run these commands on each node to install ContainerD:

```
```bash
theme={null}
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
sudo systemctl enable --now kubelet
```

Now, install ContainerD:

```
```bash
theme={null}
sudo apt-get update
sudo apt-get install -y containerd
```

Verify the installation:

```
```bash theme={null} sudo systemctl status containerd
```

```
### Step 3: Configure ContainerD for the systemd Cgroup Driver
```

It is critical that both ContainerD and kubelet use the same cgroup driver. Since systemd is the init system

```
```bash theme={null}
sudo mkdir -p /etc/containerd
containerd config default | sed 's/SystemdCgroup = false/SystemdCgroup = true/' | sudo tee /etc/containerd/config.toml
sudo systemctl restart containerd
```

For further details on cgroup drivers, please review the [container runtime documentation](#).

The screenshot shows the Kubernetes documentation page for "Configuring the container runtime cgroup driver". The left sidebar contains a navigation menu with sections like "Documentation", "Getting started", "Learning environment", "Production environment", "Container Runtimes", "Installing Kubernetes with deployment tools", "Turnkey Cloud Solutions", "Best practices", "Concepts", "Tasks", "Tutorials", "Reference", and "Contribute". The main content area has a "Note" box stating: "Starting with v1.22 and later, when creating a cluster with kubeadm, if the user does not set the `cgroupDriver` field under `KubeletConfiguration`, kubeadm defaults it to `systemd`." Below this, it explains that if you configure `systemd` as the cgroup driver for the kubelet, you must also configure `systemd` as the cgroup driver for the container runtime. It lists examples: `containerd` and `CRI-O`. A "Caution" box follows, warning that changing the cgroup driver of a Node is a sensitive operation and can cause errors. It suggests replacing the node or reconfiguring it. The page also includes sections for "Migrating to the `systemd` driver in kubeadm managed clusters" and "CRI version support", noting that Kubernetes starting v1.26 only works with v1 of the CRI API.

The screenshot shows the Kubernetes documentation page for "Configuring the kubelet cgroup driver". The left sidebar is similar to the previous one, but with "Administer a Cluster" expanded, showing "Administration with kubeadm" and "Configuring a cgroup driver". The main content area has a "Before you begin" section with links to "Migrating to the `systemd` driver", "Modify the kubelet ConfigMap", and "Update the cgroup driver on all nodes". It then explains that the Container Runtimes page explains that the `systemd` driver is recommended. A "Note" box states: "In v1.22 and later, if the user does not set the `cgroupDriver` field under `KubeletConfiguration`, kubeadm defaults it to `systemd`." It also mentions that in Kubernetes v1.28, automatic detection of the cgroup driver is available as an alpha feature. A minimal example of configuring the field explicitly is shown in a code block:

```
kubeadm - config.yaml
kind: ClusterConfiguration
apiVersion: kubeadm.k8s.io/v1beta4
kubernetesVersion: v1.21.0
...
kind: KubeletConfiguration
apiVersion: kubelet.config.k8s.io/v1beta1
cgroupDriver: systemd
```

## 4. Installing kubeadm, kubelet, and kubectl

After configuring the container runtime, install the Kubernetes components. Holding these packages prevents unintentional upgrades:

```
```bash
theme={null}
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
sudo systemctl enable --now kubelet
```
```

The components serve the following purposes:

```
* **kubeadm:** Bootstraps and manages the cluster initialization.
* **kublet:** Manages pods and containers on every node.
* **kubectl:** Provides the command-line interface to interact with the cluster.
```

```

```

### ## 5. Initializing the Kubernetes Cluster

On the master node, initialize the control plane with `kubeadm init`, making sure to specify the following:

```
* `--pod-network-cidr`: Sets the CIDR for pod networking (e.g., "10.244.0.0/16").
* `--apiserver-advertise-address`: Uses the master node's static IP.
```

Before proceeding, verify the master node's IP address:

```
```bash theme={null}
ip addr
```

For instance, if your master node's static IP is 192.168.56.11, run:

```
```bash theme={null} sudo kubeadm init --apiserver-advertise-address=192.168.56.11 --pod-network-
cidr="10.244.0.0/16" --upload-certs
```

After initialization, an `admin.conf` file is created. Configure kubectl by copying this file:

```
```bash theme={null}
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

export KUBECONFIG=/etc/kubernetes/admin.conf
```

Confirm access to the cluster by checking the pods:

```
```bash theme={null} kubectl get pods -A
```

A lack of pods in the default namespace indicates that the cluster is successfully initialized.

```

```

### ## 6. Deploying a Pod Network Add-on

To enable inter-pod communication, deploy a pod network add-on. In this demo, we use Weave Net. Run the following:

```
```bash theme={null}
kubectl apply -f [podnetwork].yaml
```

Replace `[podnetwork].yaml` with the URL or local file path to the Weave Net configuration file. This command deploys a DaemonSet that ensures the network add-on is applied across the control plane and later propagates to worker nodes.

Verify the network add-on by checking the pods:

```
```bash theme={null} kubectl get pods -A
```

```
Ensure the Weave Net pods are running on the control plane. Adjust network settings if necessary to match
```

```

```

### ## 7. Joining Worker Nodes to the Cluster

Once the pod network is deployed, add your worker nodes to the cluster using the `kubeadm join` command p

For example, run the following command on each worker node:

```
```bash theme={null}
sudo kubeadm join 192.168.56.11:6443 --token ps4rl5.0ns9vwu9exjul8tg \
  --discovery-token-ca-cert-hash sha256:fdb5c133b76f41d6d1f9ed72d90b7265de5e53a9156d7d48d83df65f3bde
```

After joining, verify the nodes from the master node:

```
```bash theme={null} kubectl get nodes
```

```
All nodes (master and workers) should appear with a "Ready" status.
```

```

```

### ## 8. Verifying the Cluster

To ensure that your Kubernetes cluster is operational, deploy a test pod (for example, an nginx container

```
```bash theme={null}
kubectl run nginx --image=nginx
kubectl get pod
```

Once the test pod is in the Running state, your cluster setup is complete. You may remove the test pod when finished.

Conclusion

This guide covered the following steps to bootstrap your Kubernetes cluster using kubeadm:

1. Reviewed VM network configurations.
2. Installed and configured ContainerD as the container runtime.
3. Installed Kubernetes components (kubeadm, kubelet, and kubectl).
4. Initialized the control plane with `kubeadm init` (including specifying pod network CIDR and the API server advertise address).
5. Deployed a pod network add-on (Weave Net).
6. Joined the worker nodes to the cluster.
7. Verified cluster functionality with a test pod deployment.

You have now successfully set up your Kubernetes cluster from scratch using kubeadm. Happy clustering!